

[Home](#) / [Blog](#) / [LangChain Ollama Integration: Complete Tutorial ...](#)

CONTENTS

[Unleash Ollama + LangChain: A
12 Minute End 2 End Tutorial](#)[Prerequisites and Environment
Setup](#)[Configuring LangChain with
Ollama: Core Integration
Methods](#)

LangChain Ollama Integration: Complete

We use essential cookies to make our site work. With your consent, we may also use non-essential cookies to improve our site, personalize content, customize advertisements, and analyze website traffic. For these reasons, we may share your data with our social media, advertising, and analytics partners. By clicking "Accept," you agree to our website's cookie use as detailed in our [Cookie Policy](#). You can change your cookie settings at any time by clicking "[Preferences](#)."

Visual workflows

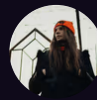
[Conclusion: Deploying and
Scaling Local AI Workflows](#)**Automate anything with
Latenode**Free forever plan · No credit card ·
5,500+ integrations[Start for free](#)

Examples

Learn how to securely
integrate local AI
workflows using a powerful
framework and model to
enhance data privacy and
operational efficiency.

Trusted by 10,000+ companies
worldwide

WRITTEN BY



Vasiliy Datsenko

Head of Customer Support

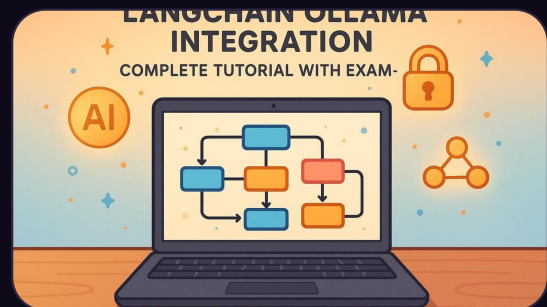
FACT CHECKED BY



Oleg Zankov

Founder and CEO

Updated June 11, 2026 ⌚ 12 min read



LangChain is a framework designed for building AI workflows, while Ollama is a platform for deploying AI models locally. Together, they enable secure, offline AI applications, ideal for industries requiring strict data privacy, such as healthcare, finance, and legal services. By running AI processes on-premises, organizations can reduce costs, eliminate reliance on cloud APIs, and

maintain control over sensitive information.

This guide walks you through setting up LangChain with Ollama for local AI workflows. From system requirements to installation steps and practical examples, you'll learn how to build text generation systems, chatbots, and document retrieval solutions - all without compromising data security. Tools like [Latenode](#) can simplify these setups further by offering a visual workflow builder, reducing the need for extensive coding. Let's explore how to create efficient, secure AI solutions tailored to your needs.

Unleash [Ollama](#) +
[LangChain](#): A 12
Minute End 2 End

Tutorial

Prerequisites and Environment Setup

Setting up LangChain with Ollama involves ensuring your system meets the necessary specifications and following proper installation steps. This preparation is key for smooth local AI development.

System Requirements

Before diving in, confirm that your development environment has enough resources to handle local model operations. Ollama loads entire models into RAM during inference, so adequate

memory is critical. For instance:

- **7B parameter models** (e.g., [Llama 3.1](#)) require at least **8 GB of available RAM**.
- **13B models** need **16 GB or more**.
- For production workloads or multiple models running simultaneously, **32 GB of system memory** is recommended.

Ollama supports **Windows 10/11**, **macOS 10.15+**, and major Linux distributions like [Ubuntu 20.04+](#), [CentOS 8+](#), and [Debian 11+](#). For faster inference, GPU acceleration is highly beneficial:

- [NVIDIA GPUs](#) require [CUDA 11.8+](#).
- [AMD GPUs](#) need [ROCm 5.4+](#).

For development and testing, CPU-only setups

work just fine.

Ensure your Python version is **3.8 or newer**, with **3.10+** preferred for compatibility with the latest LangChain releases. Storage requirements depend on the model size:

- **7B models** need 4-7 GB of space.
- **13B models** require 8-15 GB.

Using SSDs instead of traditional hard drives significantly reduces model loading times.

During initial setup, a stable internet connection is essential to download models. For example, downloading a 7B model typically takes **15-30 minutes on a 100 Mbps connection**. Once the models are downloaded, Ollama operates entirely offline, making it a great option for secure or air-gapped

environments.

Once your system is ready, you can move on to installing LangChain and Ollama.

Installing LangChain and Ollama

Start by installing the necessary Python dependencies. Use the following pip command to install LangChain along with its community integrations for Ollama:

```
pip install langchain l
```

This process should take just a few minutes.

For Ollama, installation steps depend on your operating system:

- **macOS:** Download the installer from Ollama's

official website and run the `.dmg` file.

- **Windows:** Download the executable installer and follow the setup wizard.
- **Linux:** Use the curl script:

```
curl -fsSL https://o
```

Alternatively, package managers like `apt` or `yum` can be used.

After installing Ollama, download your first model via the command line. Popular starter models include:

- **Llama 3.1 8B:** `ollama pull llama3.1:8b`
- **Mistral 7B:** `ollama pull mistral:7b`
- **Code Llama (for programming tasks):**
`ollama pull codellama:7b`

Although models are downloaded automatically when first requested, pre-downloading ensures quicker response times during initial use. Local installations also allow secure, offline operations.

To avoid dependency conflicts, create and activate a virtual environment using:

```
python -m venv langchai
```

Once installations are complete, it's time to verify that everything is working as expected.

Verifying Installations

Start the Ollama service and confirm that models are accessible. Run the following command to launch the local server (it usually binds to port

11434):

```
ollama serve
```

In a separate terminal, test the setup by running:

```
ollama run llama3.1:8b
```

This command checks if the model loads correctly and generates responses.

To verify LangChain integration, create a simple Python script that connects to your local Ollama instance. Import the necessary modules and use the Ollama LangChain wrapper to establish the connection. A successful connection will return model details, confirming proper integration.

For performance, most 7B models generate **10-30 tokens per second** on CPUs,

with GPUs boosting speeds up to three times faster.

If you encounter issues, common troubleshooting steps include:

- **Port conflicts:** Ensure no other service is using port 11434.
- **Firewall settings:** Check if your firewall is blocking the service.
- **Model file integrity:** If downloads are corrupted, re-download models using `ollama pull`.
- **Memory errors:** Insufficient RAM may require switching to a smaller model.

Latenode simplifies these workflows by handling technical complexities, ensuring your environment is ready for efficient local AI development. With installations verified, you're all

set to begin building and experimenting.

Configuring LangChain with Ollama: Core Integration Methods

LangChain and Ollama can work together to create secure, efficient local AI workflows. By connecting the right models and designing optimized prompts, you can build applications that prioritize data privacy and control, leveraging the advantages of local large language models (LLMs).

Setting Up and Configuring Ollama

Models

To begin, import the necessary classes to connect with local Ollama models.

These wrapper classes support both text completion tasks and conversational interactions:

```
from langchain_community.llms import Ollama
from langchain_community.chat_models import ChatOllama

# Basic text completion
llm = Ollama(
    model="llama3.1:8b",
    base_url="http://localhost:11434",
    temperature=0.7
)

# Chat model setup for conversational interactions
chat_model = ChatOllama(
    model="llama3.1:8b",
    temperature=0.3,
    num_predict=256
)
```

Parameters such as `temperature` influence response creativity, while

settings like `num_predict` control the maximum length of responses. Selecting the right model for your specific task is essential, as the model's capabilities can significantly impact performance and outcomes.

Customizing Prompts and Managing Outputs

Crafting clear and structured prompts is essential for obtaining effective responses from local models. Using custom prompt templates ensures that both text completion and conversational interactions are guided effectively. Here's how you can structure prompts:

```
from langchain.prompts
from langchain.schema i

# Structured prompt ten
```

```

completion_prompt = PromptTemplate(
    input_variables=["task"],
    template="""Task: {task}

Context: {context}

Please provide a detailed answer to the task.
"""

)

# Chat prompt template
chat_prompt = ChatPromptTemplate.from_messages(
    [
        SystemMessage(content="You are a helpful assistant."),
        HumanMessage(content="{task}")
    ]
)

```

By customizing prompt templates, you can fine-tune how the model processes inputs and generates outputs. This approach not only enhances the accuracy of responses but also allows you to adapt the system to specific use cases. Once prompts are tailored, you can better understand how local setups differ from cloud-based deployments.

Local vs. Cloud-

Based LLMs: Key Differences

One of the main distinctions between local and cloud-based LLMs is data handling. Local models process data on-premises, ensuring that sensitive information remains within your control. In contrast, cloud models require data to be transferred externally, which may raise privacy concerns.

Operating a local LLM setup may involve more technical configuration, but it provides complete control over model behavior, prompt management, and overall workflow customization. This level of control is particularly beneficial for organizations that prioritize both performance and security.

For those seeking to simplify local AI integrations, Latenode offers a visual workflow

builder that makes connecting to local models more intuitive. Many teams rely on Latenode for production deployments because its visual approach reduces the complexity of managing and scaling these setups. In the next section, explore how Latenode's tools streamline even the most intricate technical configurations.

sbb-itb-23997f1

Practical LangChain Ollama Examples for Workflow Automation

Setting up LangChain Ollama locally offers secure data handling and helps reduce

operational costs. Below are practical examples showcasing how LangChain Ollama can streamline workflows.

Text Completion and Q&A for Workflow Automation

This example demonstrates creating a Q&A system by combining document processing with intelligent response generation. It's particularly helpful for internal documentation, customer support, and technical troubleshooting.

```
from langchain_community
from langchain.prompts
from langchain.chains import
import json

# Initialize Ollama with
qa_llm = Ollama(
    model="mistral:7b",
    temperature=0.2, #
```

```

        num_predict=512
    )

    # Create a structured C
    qa_prompt = PromptTemp
        input_variables=["c
        template="""Based c

    Context: {context}

    Question: {question}

    Answer: Provide a direc
    )

    # Build the Q&A chain
    qa_chain = LLMChain(llm

    # Example usage with co
    company_context = """
    Our support team operat
    Emergency issues can be
    Standard response time
    """

    response = qa_chain.run
        context=company_cor
        question="What are
    )

    print(f"Response: {resp

```

The `mistral:7b` model

delivers clear and reliable answers, making it suitable for customer-facing applications that require accuracy.

Building a Chatbot for Workflow Automation

This example outlines how to create a stateful chatbot that retains conversation context, ideal for business assistance and customer support.

```
from langchain_community
from langchain.memory i
from langchain.chains i
from langchain.prompts
from langchain.schema i

# Initialize chat model
chat_llm = ChatOllama(
    model="llama3.1:8b"
    temperature=0.6, #
    num_predict=300 #
)

# Set up memory to retain
```

```

memory = ConversationBu
    k=10,
    return_messages=True
    memory_key="chat_hi
)

# Create a conversatio
conversation_prompt = (
    SystemMessage(conten
Provide clear, professi
Keep responses concise
    MessagesPlaceholder
    HumanMessage(conter
])

# Build the conversatio
conversation = Conversa
    llm=chat_llm,
    memory=memory,
    prompt=conversatio
    verbose=False
)

# Example conversation
def chat_session():
    print("Business Ass

    while True:
        user_input = ir
        if user_input.l
            print("Busi
            break

```

```

        response = conv
        print(f"Busines

# Example automated res
responses = [
    "What's our current
    "Can you remind me
    "What were the acti
]

for question in respons
    answer = conversati
    print(f"Q: {questic
    print(f"A: {answer}

```

This chatbot keeps track of conversation history, making it a practical solution for assisting teams or customers with continuity in discussions.

Integrating Retrieval Augmented Generation (RAG) for Workflow Automation

RAG combines document retrieval with text generation, enabling local models to

answer queries using extensive documentation. This is especially effective for handling technical documentation, legal materials, or research data.

```
from langchain_community
from langchain.text_splitter
from langchain_community
from langchain_community
from langchain.chains import
from langchain.prompts
import os
```

```
# Initialize Ollama connection
llm = Ollama(
    model="llama3.1:8b"
    temperature=0.3
)
```

```
embeddings = OllamaEmbedder(
    model="nomic-embed-
    base_url="http://localhost:11434"
)
```

```
# Document processing
text_splitter = RecursiveTextSplitter(
    chunk_size=1000,
    chunk_overlap=200,
    separators=["\n", "\n\n", "\n\n\n"],
)
```

```

# Process company documents
def create_knowledge_base():
    documents = []

    # Load documents from directory
    for filename in os.listdir(directory):
        if filename.endswith('.txt'):
            with open(os.path.join(directory, filename), 'r', encoding='utf-8') as f:
                content = f.read()
                document = Document(content)
                documents.append(document)

    # Split documents into chunks
    texts = text_splitter.split_documents(documents)

    # Create vector store
    vectorstore = FAISS.from_documents(texts, embedding)

    # RAG prompt template
    rag_prompt = PromptTemplate(
        input_variables=["context", "question"],
        template="""Use the following context to answer the question.
        Provide specific details where applicable.

        Context: {context}

        Question: {question}

        Answer: Based on the provided context, the answer is:
        """
    )

    # Build RAG chain

```

```

def setup_rag_chain(vectorstore, llm):
    retriever = vectorsearch.SearchRetriever(
        vectorstore=vectorstore,
        search_type="similarity",
        search_kwargs={}
    )

    rag_chain = RetrievalWrapper(
        llm=llm,
        chain_type="stuff",
        retriever=retriever,
        chain_type_kwargs={},
        return_source_documents=True
    )

    return rag_chain

# Example usage
def query_documents(rag_chain, query):
    result = rag_chain.invoke(query)

    print(f"Question: {query}")
    print(f"Answer: {result['answer']}")
    print(f"Sources: {result['sources']}")

    return result

# Sample implementation
if __name__ == "__main__":
    # Create knowledge base
    kb = create_knowledge_base()

    # Setup RAG system
    rag_system = setup_rag_chain(kb, llm)

```

```
# Query examples
queries = [
    "What is our re",
    "How do we hanc",
    "What are the r",
]

for query in queries:
    query_documents = search(query)
    print("-" * 50)
```

This method processes documents locally, enabling the creation of searchable knowledge bases without exposing sensitive data to outside platforms.

**Accelerating
Private AI
Development
with Latenode
Visual Workflows**

Latenode simplifies the process of integrating local AI models, building on the technical setup of tools like LangChain Ollama. While configuring LangChain Ollama can be complex, Latenode stands out with its visual workflows that connect seamlessly to local AI models, including Ollama. This makes private AI development more accessible, eliminating the need for intricate configurations. Below, we'll explore how Latenode's interface, cost efficiency, and workflow advantages streamline private AI development compared to traditional code-first methods.

Latenode's Visual Workflow Builder

Latenode's drag-and-drop interface transforms how users configure local AI models and manage prompts.

Instead of writing Python or JavaScript scripts to set up Ollama models, users can visually select and configure local AI models through an intuitive graphical interface.

This visual workflow builder shifts the focus from technical integration to creating and deploying secure AI workflows. Teams can easily design workflows by connecting nodes for tasks like prompt configuration and output formatting - no coding required. This approach allows for faster iterations and optimizations tailored to specific needs.

For instance, creating a document analysis workflow is straightforward: connect visual nodes like **File Upload** → **Ollama Model (Llama 3.1:8b)** → **Response Formatter** → **Database Storage**. What would typically require dozens of lines of code

is reduced to a simple, intuitive visual process.

Combining Privacy and Cost Efficiency

Latenode prioritizes privacy by ensuring all data and model interactions remain on-premises. This eliminates reliance on external APIs or cloud services, significantly reducing the risk of data exposure. Additionally, this approach can cut AI operational costs by up to 80%. For example, enterprise case studies highlight how companies have replaced \$5,000/month cloud LLM expenses with local Ollama deployments.

By abstracting complex integrations, Latenode not only enhances security but also simplifies compliance for workflows involving sensitive data. When compared to

traditional coding methods, the platform's visual workflow approach minimizes the likelihood of data leaks while streamlining operations.

Organizations often choose Latenode for production deployment of LangChain Ollama solutions because its visual workflows are easier to scale and maintain than custom-coded integrations. The platform allows teams to deploy, monitor, and update local AI workflows through its visual interface. This makes it simple to manage model updates, version workflows, and onboard new team members quickly.

Code-First vs. Visual Workflow Approaches

The distinction between traditional LangChain integration and Latenode's

visual workflow approach becomes apparent when evaluating ease of use, scalability, and team accessibility:

Feature	Lang
Ease of Use	Requ
Scalability	Manu
Team Accessibility	Limit
Onboarding Speed	Slow
Privacy Control	Full c

Latenode bridges the gap between the privacy advantages of local models like Ollama and the speed of visual development tools. This hybrid approach enables teams to build secure AI applications faster than traditional code-heavy frameworks. It supports rapid prototyping and simplifies

production maintenance, all while delivering cost savings and ensuring data stays secure.

This visual workflow approach is particularly valuable for organizations where AI projects involve cross-functional teams. Business analysts, project managers, and domain experts can actively contribute to designing AI workflows without needing deep technical expertise. By making AI workflow creation more accessible, Latenode paves the way for scalable, production-ready private AI solutions.

Conclusion: Deploying and Scaling Local AI

Workflows

Deploying integrations like LangChain and Ollama into production requires careful planning, reliable hardware, and ongoing maintenance to ensure smooth operations.

Essentials for Production Deployment

A strong foundation for local AI deployment begins with allocating the right hardware resources. Running Ollama models locally with LangChain demands adequate CPU power, RAM, and storage tailored to the workload. Regular system performance monitoring and setting up alerts for potential resource overloads or unusual activity are critical steps to ensure stability [2]. Additionally, leveraging Ollama's model

management tools for version control helps maintain consistency and reproducibility [2].

Tackling Scaling Challenges

As deployments grow, scaling introduces complexities such as hardware limitations, discrepancies in model versions, and inconsistencies in prompts. These issues can be addressed by selecting models that align with your system's capabilities, scheduling updates for LangChain and Ollama, and using structured prompt templates with built-in version control [2][1].

Organizations preparing for the future often rely on modular architectures and thorough documentation. Staying engaged with the LangChain and Ollama

communities ensures access to the latest updates and best practices. To maintain reliability at scale, teams should monitor system metrics like CPU, memory, and disk usage while implementing application-level logging to track prompt inputs and outputs.

The Case for Visual Workflows

While a code-first approach with LangChain offers unmatched flexibility, it often requires significant maintenance and technical expertise. Many teams opt for Latenode to simplify production deployment. Its visual workflows reduce the complexity of scaling and ongoing maintenance, directly addressing the challenges mentioned earlier.

Latenode's hybrid approach

blends the privacy advantages of local models like Ollama with the efficiency of visual workflow builders. This allows teams to develop secure AI applications faster, without the need for intricate configurations, making private AI development more accessible.

Advancing Local AI Development

By adopting these practices, organizations can build a strong foundation for local AI workflows. The combination of LangChain's adaptability and Ollama's local model capabilities creates a powerful platform for private AI applications. Whether you prefer the control of a code-first approach or the simplicity of visual platforms like Latenode, success lies in implementing robust monitoring, maintaining

version control, and creating workflows that can evolve with your needs.

Local AI workflows represent a transformative step toward prioritizing data privacy and cost efficiency. By pairing solid deployment strategies with Latenode's visual workflow capabilities, teams can achieve scalable, secure, and efficient AI solutions.

References

[1] <https://www.devturtleblog.com/langchain-ollama-introduction/>

[2] <https://python.langchain.com/docs/integrations/llms/ollama/>

FAQ

Frequently Asked Questions

What are the key advantages of using LangChain with Ollama for local AI workflows?



Integrating **LangChain** with **Ollama** offers several notable advantages for local AI workflows:

Stronger Data Security:

Since models operate entirely offline, your data remains secure, reducing the risk of breaches. This makes it especially suitable for businesses with strict security requirements.

Lower Costs: Companies can cut expenses by up to **80%** compared to using cloud-based APIs, making AI deployments much more affordable.

Faster Performance:

Running models locally results in quicker response times, ensuring smoother and more efficient AI processes. This integration is ideal for teams focused on maintaining privacy, reducing costs, and achieving high performance in their AI projects.

How does Latenode make it easier to integrate and manage local AI models like Ollama with LangChain?



How can I ensure data privacy and security when using LangChain with locally deployed AI models like Ollama?



Found this helpful? Share it →



WRITTEN BY

Vasiliy Datsenko

Head of Customer Support

Vasiliy Datsenko is Head of Customer Support at Latenode and a product-focused automation writer. His work connects customer conversations, workflow automation research, AI use cases, and practical product education for teams trying to automate real business processes.

[Author profile →](#)

FACT CHECKED BY

Oleg Zankov

Founder and CEO

Founder and automation product builder behind Latenode. Expert in iPaaS, AI agents, and workflow automation architecture.

[Author profile →](#)

Continue reading

LangChain Tools:...

11 min read ·
Jun 11, 2026

What is LangChain

14 min read ·
Jun 11, 2026

LangChain Python...

14 min read ·
Jun 11, 2026

Automate Smart. Grow Faster.



**GDPR
Ready**
View
report

**SOC2
Ready**
View
report

★★★★★
4.9 Capterra

★★★★★
4.7 G2

★
4

PRODUCTS	FEATURES	RESOURCES	PARTNERS	CONTACT	DOGFOODING
			Become a Partner		LinkedIn AI Agent
			Affiliate Program		Voice AI Agent
			MSP Program		Reddit AI Agent

AI Agent Builder	Visual Builder	Quick Start	ALTERNATIVES	Contact Us
MCP Server Builder	1,200+ AI Models	Documentation Blog	Why Switch vs Zapier vs Make vs n8n	Talk to Sales
RAG for AI Agents	Full-Code Node	Free Tools Academy		Book a Demo
API Management	Headless Browser	Community Forum	Clay Alternative	Support Center
Embedded iPaaS	5,500+ Integrations			Request Integration
	Templates	ABOUT		Expert Consultations
SOLUTIONS	AI Copilot NEW	Our Story		
Marketing		Pricing		
Home Services		Rewards		
HR		Security		
Customer Support				
Sales				
Finance				
Supply Chain				
Developers				
RevOps				
QA				